

Békéscsabai SZC Nemes Tihamér Technikum és Kollégium

Szakképesítés megnevezése: Szoftverfejlesztő és -tesztelő

Azonosító száma: 5 0613 12 03

VIZSGAREMEK

MaxxedOut

Készítették:

Medovarszki János	Osztály: 13.B	Oktatási azonosító: 72783053329
Tekeres Dénes	Osztály: 13.B	Oktatási azonosító: 73763502081
Teveli András	Osztály: 13.B	Oktatási azonosító: 72754991467

Tartalomjegyzék

Tartalomjegyzék	2
Bevezetés	4
Fejlesztői dokumentáció	5
Fejlesztés	5
Használt szoftverek:	5
Használt hardverek:	5
Architektúra áttekintés	5
Telepítés és beállítás.....	6
Lokális beállítás	6
Konfiguráció	7
Adatbázis leírás	9
DBMS (Database Management System).....	9
Táblák	9
Normálformák	14
E-R diagram.....	15
API dokumentáció.....	16
Alap URL	16
Hitelesítés	16
Végpontok	16
Teszt dokumentáció.....	20
Tesztelési felhasználó adatai	20
Egység tesztek (Thunder Client)	20
Funkcionális tesztek	21
Tervezett fejlesztések	24

0.2 kiadás	24
1.0 kiadás	24
Felhasználói dokumentáció.....	26
Előfeltételek	26
Felhasználói felület	26
Alapvető funkciók	26
Fő használati esetek	27
Admin felület.....	28
Alapvető funkciók	28
Fő használati esetek	28
Összegzés.....	29
Irodalomjegyzék	30

Bevezetés

Edzőterembe járás során fontos, hogy nyomon tudjuk követni eddigi teljesítményünket az edzések során. Erre létezik számos megoldás például valaki füzetbe, valaki jegyzeteibe tartja számon edzéstervét és részleteit. Viszont a mai világban nagy előnyt nyújtanak az erre kitalált alkalmazások.

Miért ezt választottuk?

Mi ezeket kutatva rájöttünk, hogy az ingyenesen elérhető applikációk nem nyújtanak teljes élményt. Ezért hoztuk létre a saját edzés követő alkalmazásunkat.

Mi a célunk ezzel?

Személyre szabott edzésterveket összeállítása, az edzésteljesítmény nyomon követése és elemzése (pl. emelt súlyok, ismétlések, időtartam). Felhasználók motivációja játékosítás révén (pl. edzés sorozatok, legjobb gyakorlatok).

Fejlesztői dokumentáció

Fejlesztés

Használt szoftverek:

- Visual Studio Code
- Visual Studio 2026
- Modern böngésző

Használt hardverek:

Olyan eszközök, amik a szükséges fejlesztői szoftverek futtatására alkalmasak.

Architektúra áttekintés

A technológiai stack főbb elemei:

- Backend: Node.js, Express, SQLite3
- Mobil: React Native, Expo
- Web: React, Vite
- Admin: .NET Windows Forms
- Adatbázis: SQLite

A backend egy Node.js alapú API, amely kezeli a logikát, az adatfeldolgozást és az adatbázis-műveleteket. Az adatok tárolása SQLite adatbázisban történik.

A rendszer három különböző klienssel kommunikál:

- egy weboldallal (React + Vite),
- egy mobilalkalmazással (React Native + Expo),
- valamint egy admin felülettel (C# Windows Forms).

Telepítés és beállítás

Lokális beállítás

A rendszer egyes komponensei külön-külön konfigurálhatók és futtathatók. Az alábbi lépések bemutatják az egyes részek telepítésének és indításának folyamatát.

Backend API

A backend beállításához először az `api` könyvtárba kell navigálni, majd telepíteni kell a szükséges függőségeket az `npm install` paranccsal.

Ezt követően létre kell hozni a környezeti változókat tartalmazó `.env` fájlt az `.env.example` másolásával. A fájlban meg kell adni a szükséges konfigurációkat, például az email szolgáltatáshoz tartozó adatokat (`EMAIL_USER`, `EMAIL_PASS`) és az API portját (`API_PORT`).

Az adatbázis inicializálása a `npm run dev` paranccsal történik, amely lefuttatja a migrációs scriptet (`db/migration.js`), és létrehozza a `maxxedout.db` SQLite adatbázist.

Végül az API szerver a `npm run dev` paranccsal indítható el.

Mobilalkalmazás (React Native)

A mobilalkalmazás beállításához az `app` könyvtárba kell navigálni, majd telepíteni kell a függőségeket.

Ezután létre kell hozni a `.env` fájlt, és be kell állítani benne a backend API és a weboldal elérési útját (`API_URL`, `WEB_URL`).

A fejlesztői szerver a `npm run dev` paranccsal indítható el, amely elindítja az Expo környezetet.

Kipróbálni, tesztelni lehet web felületen, vagy az Expo Go mobilalkalmazással a konzolban kiadott QR kód segítségével.

Weboldal (React/Vite)

A weboldal esetében a **web** könyvtárba történő navigálás után szintén a függőségek telepítésével kezdődik a folyamat.

A **.env** fájlban meg kell adni a backend API címét (**VITE_API_URL**), hogy az alkalmazás megfelelően tudjon kommunikálni a szerverrel.

A fejlesztői környezet a **npm start** paranccsal indítható el.

Admin felület (C# Windows Forms)

Az admin panel beállításához az **admin/admin** könyvtárba kell lépni, majd a **dotnet restore** paranccsal telepíteni a szükséges csomagokat.

A konfigurációhoz itt is szükséges egy **.env** fájl, amely tartalmazza az API elérési útját (**API_URL**) és az admin jogosultsághoz szükséges kulcsot (**API_KEY**).

Az alkalmazás buildelése és futtatása a **dotnet run** paranccsal történik.

Konfiguráció

Környezeti változók

A rendszer megfelelő működéséhez különböző környezeti változók beállítása szükséges. Ezek komponensenként eltérnek, és biztosítják a konfiguráció rugalmasságát, valamint az érzékeny adatok biztonságos kezelését.

Backend API

A backend működéséhez szükséges környezeti változók az email szolgáltatás konfigurációját, az admin hozzáférést és a szerver működését határozzák meg.

- **EMAIL_USER**: Az értesítések küldésére használt email cím
- **EMAIL_PASS**: Az email szolgáltatáshoz tartozó jelszó (pl. Google alkalmazásjelszó)
- **ADMIN_API_KEY**: Titkos kulcs az adminisztrátori hozzáféréshez
- **API_PORT**: Az API szerver portja (alapértelmezett: 6600)

Mobilalkalmazás

A mobilalkalmazás számára szükséges változók a backend és a webes felület elérését biztosítják.

- **API_URL**: A backend API elérési útja
- **WEB_URL**: A weboldal elérési útja

Weboldal

A weboldal konfigurációja a backend API elérhetőségére korlátozódik.

- **VITE_API_URL**: A backend API elérési útja

Admin felület

Az admin panel külön hitelesítési mechanizmust használ, ezért saját API kulcs konfigurációval rendelkezik.

- **API_URL**: A backend API elérési útja
- **API_KEY**: Adminisztrátori API kulcs a hitelesítéshez

Adatbázis leírás

DBMS (Database Management System)

A rendszerben a választott adatbázis-kezelő: **SQLite**

Előnyök

- Nincs szükség külön szerverre
- Könnyen integrálható backend alkalmazásba
- Gyors fejlesztés
- Fájlalapú működés

Hátrányok

- Korlátozott skálázhatóság
- Gyengébb párhuzamos íráskezelés

Táblák

users (felhasználók)

Ez a tábla tárolja a felhasználók alapadatait, beleértve az email címet, becenevet és jelszót.

```
CREATE TABLE users (  
    id INTEGER PRIMARY KEY NOT NULL,  
    email TEXT NOT NULL,  
    nickname TEXT,  
    password TEXT NOT NULL  
);
```

workouts (edzések)

Az edzések metaadatait tárolja, például a felhasználót, az edzés kezdő- és befejező idejét, valamint a nevét.

```
CREATE TABLE workouts (  
    id INTEGER PRIMARY KEY NOT NULL,  
    user_id INTEGER NOT NULL,  
    started_at DATETIME,  
    ended_at DATETIME,  
    name TEXT,  
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
);
```

exercises (gyakorlatok)

A táblázat a gyakorlatok nevét és típusát tartalmazza (összetett vagy izolációs).

```
CREATE TABLE exercises (  
    id INTEGER PRIMARY KEY NOT NULL,  
    name TEXT NOT NULL,  
    type TEXT NOT NULL CHECK (type IN ('Compound', 'Isolation'))  
);
```

sets (sorozatok)

A gyakorlatokhoz tartozó sorozatokat, ismétléseket és súlyokat tárolja, valamint hivatkozik az edzésre és a gyakorlatra.

```
CREATE TABLE sets (  
    id INTEGER PRIMARY KEY NOT NULL,  
    workout_id INTEGER NOT NULL,  
    exercise_id INTEGER,  
    exercise_name TEXT,  
    rep INTEGER,
```

```

weight INTEGER,

FOREIGN KEY (workout_id) REFERENCES workouts(id) ON DELETE CASCADE,

FOREIGN KEY (exercise_id) REFERENCES exercises(id) ON DELETE CASCADE,

CHECK (

    (exercise_id IS NOT NULL AND exercise_name IS NULL) OR

    (exercise_id IS NULL AND exercise_name IS NOT NULL)

)

);

```

muscle_groups (izomcsoportok)

Tárolja az összes izomcsoport nevét.

```

CREATE TABLE muscle_groups (

    id INTEGER PRIMARY KEY NOT NULL,

    name TEXT NOT NULL

);

```

muscle_groups_exercises (izomcsoport–gyakorlat kapcsolat)

Kapcsolatot hoz létre az izomcsoportok és a gyakorlatok között, és jelzi az izom szerepét (elsődleges vagy másodlagos).

```

CREATE TABLE muscle_groups_exercises (

    muscle_group_id INTEGER NOT NULL,

    exercise_id INTEGER NOT NULL,

    role TEXT NOT NULL CHECK (role IN ('Primary', 'Secondary')),

    FOREIGN KEY (muscle_group_id) REFERENCES muscle_groups(id) ON DELETE CASCADE,

    FOREIGN KEY (exercise_id) REFERENCES exercises(id) ON DELETE CASCADE

);

```

plans (edzéstervek)

A felhasználók edzésterveit tartalmazza.

```
CREATE TABLE plans (  
    id INTEGER PRIMARY KEY NOT NULL,  
    user_id INTEGER NOT NULL,  
    name TEXT NOT NULL,  
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
);
```

plans_exercises (terv–gyakorlat kapcsolat)

Egy edzéstervhez rendelt gyakorlatokat és sorozatokat tárolja.

```
CREATE TABLE plans_exercises (  
    plan_id INTEGER NOT NULL,  
    exercise_id INTEGER,  
    exercise_name TEXT,  
    sets INTEGER,  
    FOREIGN KEY (plan_id) REFERENCES plans(id) ON DELETE CASCADE,  
    FOREIGN KEY (exercise_id) REFERENCES exercises(id) ON DELETE CASCADE,  
    CHECK (  
        (exercise_id IS NOT NULL AND exercise_name IS NULL) OR  
        (exercise_id IS NULL AND exercise_name IS NOT NULL)  
    )  
);
```

codes (megerősítő kódok)

Adat változtatáskor email-re küldendő kód.

```
CREATE TABLE IF NOT EXISTS codes (  
    code TEXT PRIMARY KEY NOT NULL,  
    user_id INTEGER NOT NULL,  
    expiry INT NOT NULL,  
    FOREIGN KEY (user_id) REFERENCES users(id)  
    ON DELETE CASCADE  
);
```

refresh_tokens (frissítő tokenek)

Egy bejelentkezéskor kapott kód, amivel a hozzáférési tokent lehet frissíteni.

```
CREATE TABLE IF NOT EXISTS refresh_tokens (  
    token TEXT PRIMARY KEY NOT NULL,  
    user_id INTEGER NOT NULL,  
    expiry INT NOT NULL,  
    FOREIGN KEY (user_id) REFERENCES users(id)  
    ON DELETE CASCADE  
);
```

access_tokens (hozzáférési tokenek)

Egy bejelentkezéskor kapott kód, amivel az api kéréseket lehet érvényesíteni.

```
CREATE TABLE IF NOT EXISTS access_tokens (  
    token TEXT PRIMARY KEY NOT NULL,  
    user_id INTEGER NOT NULL,  
    expiry INT NOT NULL,  
    FOREIGN KEY (user_id) REFERENCES users(id)  
    ON DELETE CASCADE  
);
```

Normálformák

1. Normálforma (1NF)

Minden mező atomi értékeket tartalmaz (nincs tömb vagy ismétlődő csoport).

Minden rekord egyértelműen azonosítható elsődleges kulccsal (id vagy összetett kulcs).

Az adatok oszloponként egységes típusúak.

2. Normálforma (2NF)

Minden nem kulcs attribútum teljes mértékben függ az elsődleges kulcstól.

Azoknál a tábláknál, ahol összetett kulcs lehetne (pl. kapcsolótáblák), nincs részleges függőség.

A kapcsolatok külön táblákba (pl. muscle_groups_exercises, plans_exercises) kerültek, így megszűntek a részleges függőségek.

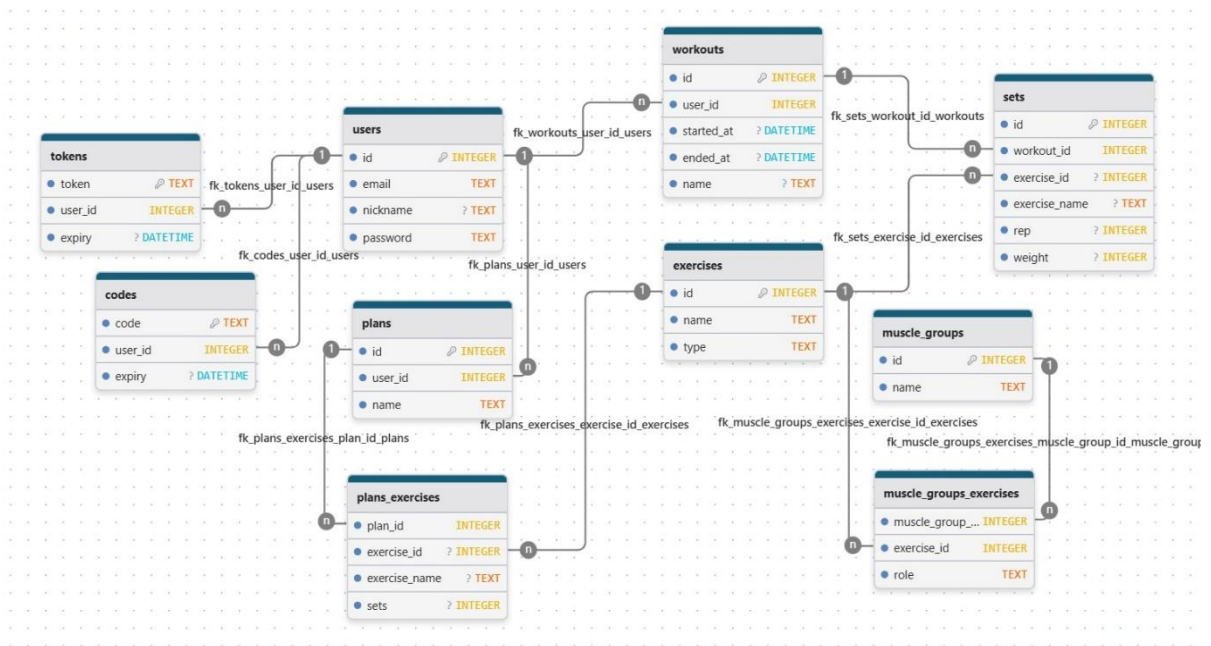
3. Normálforma (3NF)

Nincsenek tranzitív függőségek, azaz egy nem kulcs mező nem függ más nem kulcs mezőtől.

Például a felhasználói adatok külön (users) táblában vannak, és más táblák csak hivatkoznak rájuk idegen kulccsal.

Az ismétlődő adatok (pl. gyakorlatok, izomcsoportok) külön entitásokba lettek szervezve.

E-R diagram



API dokumentáció

Alap URL

http://localhost:6600

Hitelesítés

- Az API végpontok token-alapú hitelesítést használnak.
- Két fajta token van, egy frissítő és egy hozzáférési.
- A hozzáférési 10 percig él, ezután mindkettőből újat kap a felhasználó, a frissítési token validálása után.
- Ezáltal kijelentkeztetés nélkül, de biztonságosan használhatja a felhasználó az alkalmazást.

Végpontok

Authentikáció

Végpont	Módszer	Leírás	Hitelesítés
/register	POST	Új felhasználó regisztrálása	Nem
/login	POST	Felhasználó bejelentkezése	Nem
/forgot-password	POST	Jelszó visszaállítás kérése	Nem
/reset-password	POST	Jelszó visszaállítása kóddal	Nem
/verify-code	POST	Jelszó-visszaállító kód ellenőrzése	Nem

Felhasználók

/user	PATCH	Felhasználói adatok frissítése	Igen (User)
/user	DELETE	Felhasználói fiók törlése	Igen (User)

/user/admin	GET	Összes felhasználó lekérdezése	Igen (Admin)
/user/admin	POST	Új felhasználó hozzáadása	Igen (Admin)
/user/admin	PUT	Felhasználó frissítése	Igen (Admin)
/user/admin/:id	DELETE	Felhasználó törlése	Igen (Admin)

Edzések

Végpont	Módszer	Leírás	Hitelesítés
/workouts	GET	Edzések lekérdezése	Igen (User)
/workouts/:id	GET	Egy konkrét edzés lekérdezése	Igen (User)
/workouts	POST	Új edzés hozzáadása	Igen (User)
/workouts/:id	DELETE	Egy edzés törlése	Igen (User)

Edzéstervek

Végpont	Módszer	Leírás	Hitelesítés
/plans	GET	Felhasználó edzés terveinek lekérése	Igen (User)
/plans/info/:id	GET	Edzésterv részleteinek lekérése	Igen (User)
/plans/:id	GET	Egy konkrét edzésterv lekérdezése	Igen (User)
/plans	POST	Új edzésterv hozzáadása	Igen (User)

/plans/:id	PATCH	Edzésterv frissítése	Igen (User)
------------	-------	----------------------	-------------

/plans/:id	DELETE	Edzésterv törlése	Igen (User)
------------	--------	-------------------	-------------

Statisztikák

Végpont	Módszer	Leírás	Hitelesítés
---------	---------	--------	-------------

/statistics	GET	Felhasználói statisztikák lekérése	Igen (User)
-------------	-----	------------------------------------	-------------

Gyakorlatok

Végpont	Módszer	Leírás	Hitelesítés
---------	---------	--------	-------------

/exercises	GET	Összes gyakorlat lekérdezése	Nem
------------	-----	------------------------------	-----

/exercises/:id	GET	Egy konkrét gyakorlat lekérdezése	Nem
----------------	-----	-----------------------------------	-----

/exercises/admin	GET	Összes gyakorlat lekérdezése	Igen (Admin)
------------------	-----	------------------------------	--------------

/exercises/admin	POST	Új gyakorlat hozzáadása	Igen (Admin)
------------------	------	-------------------------	--------------

/exercises/admin	PUT	Gyakorlat frissítése	Igen (Admin)
------------------	-----	----------------------	--------------

/exercises/admin/:id	DELETE	Gyakorlat törlése	Igen (Admin)
----------------------	--------	-------------------	--------------

Izomcsoportok

Végpont	Módszer	Leírás	Hitelesítés
---------	---------	--------	-------------

/muscle_groups	GET	Összes izomcsoport lekérdezése	Nem
----------------	-----	--------------------------------	-----

/muscle_groups/admin	POST	Új izomcsoport hozzáadása	Igen (Admin)
/muscle_groups/admin	PUT	Izomcsoport frissítése	Igen (Admin)
/muscle_groups/admin/:id	DELETE	Izomcsoport törlése	Igen (Admin)

Teszt dokumentáció

Tesztelési felhasználó adatai

E-mail cím: johndoe@yahoo.com

Jelszó: 1234

Egység tesztek (Thunder Client)

Authentikáció

Leírás	Előfeltétel	Lépések	Várt eredmény
Bejelentkezés	Felhasználó létezik az adatbázisban	Érvényes email/jelszó	Felhasználói adatok visszaadása
Védett API kérés élő tokennel	Felhasználónak élő hozzáférési tokenje van	Bármilyen védett API kérés (pl.: edzéstervek)	Edzésterv visszaadása
Védett API kérés lejárt tokennel	Felhasználónak lejárt hozzáférési tokenje van	Bármilyen védett API kérés (pl.: edzéstervek)	„Expired token” hiba
Token frissítés élő frissítési tokennel	Felhasználónak élő frissítési tokenje van	Token frissítési útvonal hívása a frissítési tokennel	Visszaadja az új hozzáférési és frissítési token
Token frissítés lejárt frissítési tokennel	Felhasználónak lejárt frissítési tokenje van	Token frissítési útvonal hívása a frissítési tokennel	„Unauthorized” hiba

Funkcionális tesztek

Felhasználó hitelesítés

Leírás	Előfeltétel	Lépések	Várt eredmény
Sikeres bejelentkezés	Felhasználó létezik az adatbázisban	Érvényes email/jelszó → „Bejelentkezés”	Átirányítás a dashboardra, adatok mentve
Sikertelen bejelentkezés (rossz jelszó)	Felhasználó létezik	Hibás jelszó → „Bejelentkezés”	„Invalid credentials” hiba
Elfelejtett jelszó folyamat	Felhasználó létezik	„Elfelejtett jelszó” → email → kód → új jelszó	Email elküldve, jelszó frissítve
Hibás email - formátum		Hibás email (pl. „test”) → „Bejelentkezés”	„Invalid email” hiba

Edzéstervek kezelése

Leírás	Előfeltétel	Lépések	Várt eredmény
Új edzésterv létrehozása	Felhasználó be van jelentkezve	Gyakorlatok kiválasztása → “Save”	Edzésterv mentve, megjelenik a listában
Edzés szerkesztése	Edzés létezik	“Edit plan” → Szerkesztés → “Save”	Változások mentve DB-ben és UI-ban
Edzés törlése	Edzés létezik	Megnyitás → „Törlés”	Edzés eltávolítva

Edzés követés

Leírás	Előfeltétel	Lépések	Várt eredmény
Új edzés létrehozása	Felhasználó be van jelentkezve	Edzés elemeinek beírása → mentés	Edzés mentve, megjelenik a listában
Edzés törlése	Edzés létezik	Megnyitás → „Törlés”	Edzés eltávolítva
Szűrés dátum szerint	Több edzés létezik	„Múlt hónap” kiválasztása	Csak adott időszak edzései

Gyakorlatok és izomcsoportok kezelése

Leírás	Előfeltétel	Lépések	Várt eredmény
Új gyakorlat hozzáadása (Admin)	Admin bejelentkezve	„Hozzáadás” → űrlap → mentés	Gyakorlat bekerül az adatbázisba
Izomcsoport hozzárendelés	Gyakorlat létezik	Szerkesztés izomcsoportok mentés	→ Kapcsolat mentve → DB-ben
Gyakorlat keresése	Több gyakorlat létezik	„Bench” beírása	Szűrt lista
Izomcsoport törlése	Izomcsoport létezik	Kiválasztás → törlés	Törölve DB-ből

Statisztika

Leírás	Előfeltétel	Lépések	Várt eredmény
Összes edzésidő megtekintése	Van edzés	„Statisztika” → „Összes idő”	Helyes összeg
Legjobb teljesítmények	Van edzés	„Best lifts” ellenőrzése	Max értékek helyesek
Havi streak	Folyamatos edzés	„Streak” ellenőrzése	Helyes szám

Admin panel

Leírás	Előfeltétel	Lépések	Várt eredmény
Új felhasználó hozzáadása	Admin bejelentkezve	„Add User” → mentés	Felhasználó létrejön
Felhasználónév módosítása	Felhasználó létezik	Szerkesztés → mentés	Frissítve

Tervezett fejlesztések

0.2 kiadás

Offline mód

- Edzéstervek, előző edzések tárolása
- Edzés elmentése eszközre

Edzés becsült befejezési ideje

- A felhasználó kaphat egy becsült időt, mikor fejezi be edzését

1.0 kiadás

Felhasználói felület (UI) fejlesztése

- Teljes UI megújítás modernebb dizájnnal
- Swipe alapú navigáció bevezetése a gyorsabb kezelhetőség érdekében
- Animált betöltési elemek alkalmazása a vizuális élmény javítására

Felhasználói fiók kezelés bővítése

- E-mail cím módosításának lehetősége

Funkcionális egyszerűsítés

- Idézetek funkció eltávolítása

Rang- és XP rendszer bevezetése

A felhasználók motiválása és fejlődésük nyomon követése érdekében egy kombinált rang- és tapasztalati pontrendszer kerül kialakításra.

- XP számítása:
 - Összes megmozgatott súly
 - Edzések rendszeressége
 - Progresszíven nehezedő szintlépési rendszer

- Rangok meghatározása az elért szintek alapján
- Statisztikai nézetben vizuális megjelenítés

Statisztikai modul bővítése

- Személyes rekordok (PR) megjelenítése gyakorlatonként

Felhasználói dokumentáció

A MaxxedOut egy fitness alkalmazás, amely segít a felhasználóknak edzésprogramjaik nyomon követésében és elemzésében. Az alkalmazás zökkenőmentes élményt nyújt, lehetővé téve a felhasználók számára, hogy személyre szabott tervek segítségével könnyedén bonyolíthassák le edzéseiket.

Előfeltételek

Az alkalmazás használatához szükséges

- egy okostelefon
- internetkapcsolat
- regisztrált felhasználói fiók

A telepítőt a GitHubról lehet letölteni (később webáruházból is) és regisztrálni a felhasználói fiókot.

Felhasználói felület

Alapvető funkciók

Edzéstervek

- Személyre szabott edzéstervek készítése és követése.

Gyakorlat adatbázis

- Gyakorlatok böngészése izomcsoport és típus szerint (összetett/izolációs).
- Egyéni gyakorlatok hozzáadása.

Edzésnaplózás

- Edzések rögzítése gyakorlatokkal, sorozatokkal, ismétlésekkel és súlyokkal.
- Az edzés időtartamának és a pihenőidők nyomon követése.

Statisztikák és elemzések

- Edzéstörténet és egyéni rekordok megtekintése.
- Mutatók követése, például az összes megemelt súly, átlagos edzésidő és heti a megjelenés.

Értesítések és emlékeztetők

- Push értesítések pihenési idő lejártakor.
- E-mail küldése megerősítő kódokhoz.

Felhasználói profilok

- Felhasználói profilok létrehozása és kezelése becenevekkel.
- Személyes adatok frissítése (pl. becenév, e-mail, jelszó).

Fő használati esetek

Edzésterv létrehozása

1. A felhasználó megnyitja az edzéstervek szekciót
2. Rányom a plusz gombra
3. Hozzáad gyakorlatokat, szetteket
4. A rendszer elmenti a tervet, és lehetővé teszi annak betöltését
5. A felhasználó szükség szerint módosíthatja a tervet

Edzés rögzítése

1. Kiválasztja az előre megírt edzéstervet
2. Megadja az edzés adatait (gyakorlatok, szettek, ismétlések, súlyok)
3. A rendszer elmenti az adatokat az adatbázisba

Előző edzések megtekintése

1. A felhasználó megnyitja a napló szekciót
2. Kiválaszt egy előző edzést a korábbi edzésekből, vagy egy pontos dátumot a naptárból

Admin felület

Alapvető funkciók

Gyakorlatok felvitele

- Egy gyakorlat információinak megadásával felvinni, majd adminisztrálni őket

Felhasználók adminisztrálása

- A felhasználók adatainak módosítására, regisztrálására és törlésére van lehetőség

Fő használati esetek

Izomcsoportok adminisztrálása

1. Az adminisztrátor megnyitja az izomcsoportok szekciót
2. Hozzáad / módosít / töröl izomcsoportokat

Gyakorlat felvitele

1. Az adminisztrátor megnyitja a gyakorlatok szekciót
2. Gyakorlat típus kiválasztása
3. Gyakorlat nevének beírása
4. Izomcsoport(ok) megadása (opcionális)
5. Rányom az „Add” gombra

Összegzés

A MaxxedOut projekt fejlesztése során nemcsak szakmai tapasztalatot szereztünk, hanem betekintést kaptunk abba is, hogyan működik egy komplex, többplatformos alkalmazás teljes fejlesztési folyamata. A projekt elkészítése közben megtapasztaltuk a csapatmunka fontosságát, a közös problémamegoldást, valamint azt, hogy mennyire lényeges a megfelelő tervezés és kommunikáció egy szoftverfejlesztési projektben.

A fejlesztés során számos új technológiát és megoldást ismertünk meg, különösen a backend API-k, az adatbázis-kezelés, a mobilfejlesztés és a felhasználói hitelesítés területén.

Fontos szerepet kapott a dokumentációk elkészítése, a tesztelés, az adatbázis megfelelő felépítése, valamint a felhasználói élmény figyelembevétele is. Emellett sok tapasztalatot szereztünk a hibakeresésben és az alkalmazások optimalizálásában.

Úgy gondoljuk, hogy a MaxxedOut nemcsak egy sikeresen elkészített vizsgaprojekt, hanem egy olyan munka is, amely jól tükrözi az eddig megszerzett tudásunkat és fejlődésünket.

Irodalomjegyzék

React Native – Enviroment Setup

<https://reactnative.dev/docs/environment-setup>

React Native – React Native Notifications

<https://wix.github.io/react-native-notifications/docs/getting-started/>

React Native – React Native Calendars

<https://www.npmjs.com/package/react-native-calendars>

React Native – Bottom Tab Navigation

<https://reactnavigation.org/docs/bottom-tab-navigator/>

React Native - Async Storage

<https://react-native-async-storage.github.io/3.0/>

React Native - createContext

<https://react.dev/reference/react/createContext>

MDN Web Docs - Javascript tömb mutáció

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/from

Nodejs - Dotenv

<https://www.npmjs.com/package/dotenv/v/17.4.2>

Expo – Expo Constants

<https://docs.expo.dev/versions/latest/sdk/constants/>

Expo - Build

<https://docs.expo.dev/build/introduction/>

Ionicons - Icons

<https://ionic.io/ionicons/usage>

Nodemailer

<https://nodemailer.com/>